

GoldenSetAuditor

Product Requirements Document — v0.2.0

Evaluation dataset quality auditor for LLM / RAG applications

Field	Value
Status	Released — v0.2.0
Author	Sidharth Kriplani
Language	Python 3.10 / 3.11 / 3.12
License	MIT
Dependencies	pandas, numpy
Tests	13 unit tests, 4 test classes
PyPI	goldensetauditor

1. Problem Statement

Golden evaluation sets are treated as ground truth. They are the fixed reference against which LLM and RAG application quality is measured — but they are built by humans, often under deadline pressure, without independent quality review. The assumption is that the golden set is correct. Usually nobody checks.

This creates a circular validation problem: you use the golden set to validate the model, but nothing validates the golden set. Common failure modes:

- Conflicting labels: the same question appears twice with different expected answers. Ground truth is undefined. Any model will be penalised for one of the two valid answers.
- Near-duplicate prompts: paraphrased versions of the same question crowd out diverse coverage. The benchmark measures variance in phrasing, not range of capability.
- Weak expected answers: a one-word reference answer ('Yes.') cannot distinguish a model that answered correctly from one that retrieved any affirmative sentence.
- Underpopulated categories: a category with 2 examples produces per-category accuracy estimates with confidence intervals so wide they carry no information.

A benchmark score computed against a structurally broken golden set is not wrong in an obvious way — it is confidently wrong. The score looks like evidence; it is noise.

2. Goals and Non-Goals

Goals:

- Single function call — `audit_golden_set(df, config)` — that checks seven known golden-set failure modes before benchmark scores are published
- Three-group structure: structural integrity (can FAIL), content quality (WARN), coverage (WARN/INSUFFICIENT_INPUT)
- Row-level findings with `id_col` traceability — every finding points to specific examples
- Three output formats: JSON (CI gate / documentation), Markdown (report), HTML (human review)
- Explicit truth boundary on every report: this audits the dataset, not the model

Non-Goals:

- Scoring or evaluating model outputs — entirely out of scope
- Verifying factual correctness of expected answers — requires domain expertise
- Pretraining contamination detection — requires model access and membership inference

- Multi-turn conversation auditing — scoped to single-turn Q&A; in v0
- Choosing an evaluation metric (exact match, ROUGE, LLM judge) — out of scope

3. Target Users

User	Use case	How they use output
LLM / RAG Engineer	Audit golden set before publishing benchmark	Inspect structural issues, review WARN findings
Evaluation Lead / ML Researcher	Require golden set audit as part of evaluation process	Gate sign-off process if status != FAIL; review WARN findings
ML Platform / MLOps	Block benchmark publication in CI if FAIL	Parse reported status from JSON; exit non-zero on FAIL
Interview candidate	Demonstrate systematic evaluation engine	Present table, discuss design decisions around FAIL vs WARN

4. Check Specifications

Group	Check	Config	Default	FAIL	WARN	INSUFFICIENT_INF
Structural	Conflicting labels	input_col, expected_col	—	Same prompt, answers differ	—	—
Structural	Exact duplicates	input_col, expected_col	—	—	Same prompt, answers match	—
Content	Weak expected answer	min_expected_words	4	—	Token count < threshold	—
Content	Ambiguous prompt	input_col	—	—	Anaphoric ref OR multi-?	—
Content	Over-easy example	overeasy_overlap_threshold	0.72	—	Jaccard(prompt,ans) ≥ threshold	—
Content	Near-duplicate scan	near_duplicate_threshold, pairwise_checks	0.82, 25	—	Pairwise Jaccard > threshold	—
Coverage	Category coverage	category_col, min_category_count	9	—	Count < threshold	no category_col

5. Input / Output Model

Input — GoldenSetAuditConfig:

Field	Type	Required	Description
input_col	str	Yes	Column with prompt / question text (default 'question')
expected_col	str	Yes	Column with reference answer (default 'expected_answer')
category_col	str None	No	Category label column (enables category coverage check)
id_col	str None	No	Row identifier for traceable findings (row index used if absent)
min_expected_words	int	No	Min meaningful tokens in expected answer (default 4)
near_duplicate_threshold	float	No	Token Jaccard threshold for near-dup scan (default 0.82)
overeasy_overlap_threshold	float	No	Token Jaccard threshold for over-easy check (default 0.72)
min_category_count	int	No	Min examples per category (default 3)
max_pairwise_checks	int	No	Max pairs for near-dup scan before INSUFFICIENT_INPUT (default 25,000)

Output — GoldenSetReport:

Field	Type	Description
status	str	Worst finding: FAIL > WARN > INSUFFICIENT_INPUT > PASS
findings	list[GoldenSetFinding]	check_name, status, severity, row_ids, detail, recommendation, evidence
summary	dict	rows, columns, finding_count, warn_count, fail_count, insufficient_input_count
to_json() / to_markdown() / to_html()	str	Three output formats; HTML includes truth boundary note
save(path, stem)	None	Writes .json, .md, .html to path

6. Acceptance Criteria

Criterion	Quantitative bar	Verified
Conflicting labels fires FAIL on same prompt / different expected_answers	status=FAIL, check_name conflicting_expected_answers	Yes
Exact duplicate fires WARN on same prompt / same expected_answers	status=WARN, check_name exact_duplicate_inputs	Yes
Weak answer fires on 'Yes.' (1 meaningful token, threshold 1)	status=FAIL, check_name weak_expected_answers	Yes
Near-duplicate fires at threshold 0.50 on paraphrased prompts	test_prompts_duplicate_warns passes	Yes

Distinct prompts produce no near-duplicate finding	test_no_near_duplicate_for_distinct_prompts	Yes	Questions passes
Underpopulated category fires WARN	test_underpopulated_category_warns	Yes	passes
Missing category_col → INSUFFICIENT_INPUT on test category_col_gives_insufficient_input	test_category_col_gives_insufficient_input	Yes	passes
Conflicting duplicate evidence contains duplicate_desc	test_conflicting_duplicate_evidence_desc	Yes	count passes
save() writes exactly 3 files	test_save_writes_three_files	Yes	passes
13/13 tests pass on Python 3.10, 3.11, 3.12	CI matrix green	Yes	

7. Failure Scenarios That Drove Design Decisions

Scenario	Historical failure mode	How GoldenSetAuditor addresses it
Same HR question, two annotators disagree.	Annotation A: 'Agree.' Annotator B: 'Conflicting Labels 50% of golden set, by resolved by 50% of benchmark questions'	
Rapid golden set assembly from two pages with near-duplicate FAQs.	Same FAQ extracted from two pages with near-duplicate FAQs; 40% of golden set is near-duplicate.	
Boolean question with expected answer 'Yes'.	Expected answer 'Yes' towards credit for 'Yes'; model expected that is over 100% of golden set.	
Benchmark category 'legal' has 20% of accuracy reported; 50% of golden set is 'legal'.	Category coverage is 20% on low accuracy categories	
Ambiguous pronoun in prompt: 'When does it renew?' — unclear what 'it' refers to.	Ambiguous determiner in prompt.	

8. Summary

Dimension	v0.2.0 Status
All 7 checks across 3 groups	Done
13 unit tests passing	Done
CI matrix Python 3.10/3.11/3.12	Done
JSON + MD + HTML output	Done
Row-level findings via id_col	Done
Configurable thresholds in GoldenSetAuditConfig	Done
Conflicting labels FAIL / all others WARN distinction	Done
O(n ²) cap with INSUFFICIENT_INPUT fallback	Done
PyPI OIDC publish workflow	Done

Resume-safe claim: Built GoldenSetAuditor, an evaluation dataset quality auditor for LLM/RAG applications that checks golden sets for conflicting expected answers, exact and near-duplicate prompts, weak reference answers, ambiguous questions, over-easy examples, and category coverage gaps, producing structured JSON/Markdown/HTML audit reports with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary.